Network Packet Analyzer Using Python

Mini Project Report

Submitted by

Sathish Kumar and Team

Register numbers and additional team members to be filled

Submitted in partial fulfilment of the requirements
for the award of the degree of

Bachelor of Engineering
in
Computer Science and Engineering

Department of Computer Science and Engineering

Adhiparasakthi Engineering College

Melmaruvathur Tamil Nadu

Academic Year 2026 to 2027

Submission Date _____

Certificate

Unsigned certificate template

This page is reserved for the college to complete and approve after reviewing the project and report. The names, register numbers, guide details and signatures below have not been supplied.

This is to certify that the mini project report entitled Network Packet Analyzer Using Python, submitted by Sathish Kumar and the team members listed below, has been reviewed for the Computer Networks mini project in the Department of Computer Science and Engineering.

Student name	Register number
Sathish Kumar	_____
_____	_____
_____	_____
_____	_____

Project Guide Name and Designation _____

Review Date _____ Place _____

Signature of Project Guide **Signature of Head of Department**

_____ _____

Internal Examiner **External Examiner**

_____ _____

The college may replace this template with its prescribed certificate wording before submission.

Declaration and Submission Details

Team declaration

We submit this report for the mini project Network Packet Analyzer Using Python. It describes the included Python implementation, the synthetic demonstration capture, the test methods and the results observed during verification. External libraries, protocol specifications and documentation are acknowledged in the references.

The reported packet counts come from the bundled demonstration dataset. They do not represent monitoring of a production network or a benchmark of capture throughput. The application interface was exercised using saved packets. Live capture on the target Windows laptop and its Npcap setup require a final local check.

We will complete the student and guide details below and review this declaration before signing the final college submission.

Submission field	Details
Project title	Network Packet Analyzer Using Python
Team representative	Sathish Kumar
Department	Computer Science and Engineering
Academic year	2026 to 2027
Additional members	To be filled by the team
Guide	To be filled by the team

Acknowledgement

The team acknowledges the Scapy, Python and Wireshark communities for the software and technical documentation used in this implementation. The official protocol specifications provide the basis for explaining the observed packet fields. The team may add the names of the faculty and other people who supported the project after confirming their details.

Student signatures _____ Date _____

Abstract

Network Packet Analyzer Using Python is a desktop application for studying packet headers and traffic summaries. It connects Computer Networks topics such as TCP/IP, IP addressing, transport ports and protocol encapsulation to packets that students can inspect. The application uses Python for its logic, Scapy for packet processing and Tkinter for the desktop interface. Wireshark is used as an independent reference tool for saved captures.

The system accepts live packets from a selected interface and saved PCAP or PCAPNG files. It extracts source and destination addresses, TCP or UDP ports, protocol labels, timestamps and captured lengths. Users can combine protocol, IP or CIDR, port and text conditions to select packets. A packet inspector shows Scapy's decoded fields and a limited hexadecimal preview. Statistics summarize retained packets, captured bytes, protocol counts and frequently observed addresses. The current matching view can be exported as CSV or classic PCAP.

A background worker handles capture and file reading while the Tkinter thread owns the display and session store. A bounded queue limits pending packet handoff. Retention stops at 5,000 packets or 32 MiB of captured bytes. The application reports queue drops and skipped decoding attempts separately so that the displayed counters have a clear scope.

The bundled synthetic capture contains 37 packets and 2,384 captured bytes. All 29 automated cases passed. Desktop checks covered loading, selection, filters, views and exports. Independent offline verification with TShark checked ten filter counts and compared addresses, lengths and available transport ports across all 37 frames. Live capture and Windows-specific driver behavior remain subject to verification on the target laptop. The delivered system supports repeatable protocol demonstrations without claiming encrypted-content inspection or lossless high-rate capture.

Keywords Python Scapy Wireshark TCP/IP packet analysis Tkinter PCAP

Chapter 1 Introduction

Project overview

A network application presents useful content to its user, but the packets carrying that content remain hidden during ordinary use. A web request can involve name resolution, transport connection setup and multiple data packets. Observing those packets helps students relate protocol theory to fields such as an IP address, a port or a TCP flag.

This project provides a focused packet-analysis application with readable Python modules. A student can open a known capture, select a packet and identify the layers carried inside it. The same interface also supports short live sessions when the operating system and capture driver permit them. A saved synthetic dataset gives the team a repeatable starting point for demonstrations.

Problem statement

Students often learn TCP/IP concepts as separate definitions. They need a practical way to inspect how addresses, protocols, ports and lengths appear together in an actual packet representation. A small implementation should expose that processing clearly, allow targeted filtering and produce files that another analyzer can reopen.

Objectives

- Capture packets from one selected interface or stream packets from a saved capture file.
- Extract common network and transport fields and present them in a packet table.
- Combine display filters without modifying the retained source packets.
- Inspect decoded protocol layers and the captured byte representation.
- Summarize retained traffic and export the matching view for comparison.
- Keep demonstrations reproducible through synthetic input and automated checks.

Project scope

The implemented scope is educational packet inspection. The application does not infer an application protocol merely from a familiar port. It does not reconstruct application conversations, decrypt encrypted

payloads or claim complete visibility of a switched network. The results chapter states which behaviors were verified and which still depend on the target laptop.

Chapter 2 Networking Fundamentals

TCP IP layers and encapsulation

In the TCP/IP model, application messages pass through transport and internet processing before the link layer carries them. Captured Ethernet frames can therefore contain an IP packet, which contains a TCP segment or UDP datagram, which may contain an application message. The analyzer reads these decoded layers instead of treating the whole capture as a string.

Layer	Example	Fields observed in this project
Application	DNS	Query name, message identifier and query or response flag
Transport	TCP and UDP	Source port, destination port and TCP control flags
Internet	IPv4 and IPv6	Source and destination IP addresses and network version
Link	Ethernet and ARP	MAC addresses or ARP protocol addresses

Addresses and ports

IPv4 uses 32-bit addresses and IPv6 uses 128-bit addresses. An IP address identifies a network endpoint or interface, while a TCP or UDP port identifies a transport endpoint used by an application. A single host can therefore exchange packets for several applications at once. The IP filter accepts an exact address or a CIDR network and checks both packet endpoints. [4] [5]

Transport and control protocols

TCP supplies an ordered, reliable byte-stream service using connection state and control information. SYN and ACK flags are useful examples for discussing connection setup. UDP sends datagrams without TCP's reliability and ordering mechanisms. DNS can use UDP or TCP, so the analyzer retains a separate transport field even when the main label is DNS. [6] [7] [8]

ICMP and ICMPv6 carry control and diagnostic messages. ARP resolves an IPv4 protocol address to a link-layer address on the local link. These packets do not have TCP or UDP ports, so the table displays a dash in the ports column rather than inventing a value.

Captured lengths and time

The byte metric is the sum of bytes retained for captured packets. It is not the size of the PCAP file and need not equal all physical-link overhead. The capture span uses the largest retained timestamp minus the smallest timestamp, which also handles files whose records are out of timestamp order.

Chapter 3 System Analysis

Existing tools

Wireshark already provides detailed packet inspection, protocol decoding and filtering. It is appropriate for professional analysis and for independently examining this project's output. The reason for implementing a smaller analyzer is to make the data path and program structure understandable in a college mini project. The project complements an established tool with a compact Python implementation. [3]

Aspect	Wireshark	Implemented mini project
Primary use	General packet investigation	Focused protocol-learning demonstration
Implementation visible to the team	Large established codebase	Small Python modules supplied in the package
Filters	Extensive display and capture syntax	Protocol, endpoint, port and metadata controls
Packet files	Many formats and advanced workflows	PCAP/PCAPNG input and classic PCAP output
Advanced analysis	Stream and protocol analysis features	Header inspection and basic summaries

Proposed system

The proposed system has a desktop interface and an optional terminal mode. A source-selection layer starts live capture or saved-file reading. Each packet moves through a bounded queue to the GUI thread. Metadata records are retained within fixed limits. Display filters select records for the table and export operations, while statistics continue to describe the complete retained session.

Feasibility

The application uses a standard Python GUI toolkit and a single pinned third-party dependency. Saved-file analysis can run without an administrator account or a capture driver, which supports classroom demonstrations. Live operation adds the platform's capture permissions and, on Windows, Npcap. No server, database or paid service is required by the implementation.

Success criteria

A successful demonstration loads the bundled capture, displays its known fields, returns the expected filtered counts, preserves captured bytes in a PCAP round trip and matches an independent analyzer on selected fields. These criteria are verifiable. No target for packets per second is asserted because a throughput benchmark was not performed.

Chapter 4 Requirements

Software requirements

Component	Requirement or role
Python	Python 3.10 or later; verification used Python 3.12.14
Scapy	Version 2.8.0 pinned in requirements.txt
Tkinter	Desktop widgets supplied with an appropriate Python installation
Npcap on Windows	Required for live capture; permissions depend on its settings
libpcap on Unix	Needed for BPF capture-filter support
Wireshark or TShark	Independent offline verification; optional at application runtime
Operating system	Windows, Linux or macOS with a compatible graphical Python setup

Practical hardware recommendation

A general-purpose laptop with about 4 GB of RAM, a working display and sufficient disk space for Python and capture files is a reasonable classroom starting point. This is a recommendation, not a measured minimum. Large captures and Scapy packet objects increase memory consumption. The application's captured-byte bound does not bound the entire Python process to 32 MiB.

Functional requirements

- Provide demo loading, saved-file opening, interface selection and capture start/stop controls.
- Display packet metadata and allow users to inspect the selected packet.
- Validate filter input and export matching records without changing the source capture.
- Expose traffic summaries and indicate application queue drops and decode skips.

Operational requirements

Capture and reading must run outside the Tkinter event loop. A full handoff queue must not grow without limit. Saved-file reading waits for queue space, while live acquisition counts packets that cannot enter the queue. The interface must remain usable when a capture fails, a file is invalid or the retention limit is reached. Exports must not overwrite a destination with an incomplete write.

Use an interface and traffic that the team is permitted to inspect. The supplied demo avoids this dependency because it writes constructed packets to a local file without transmitting them. Scapy's platform instructions explain the capture prerequisites. [2]

Chapter 5 System Architecture

The same decoder and session model support the desktop interface and terminal analysis. In the desktop path, the background worker performs input operations and the Tkinter thread owns the visible state. This separation lets the user stop an import or interact with retained records while packet acquisition is in progress.

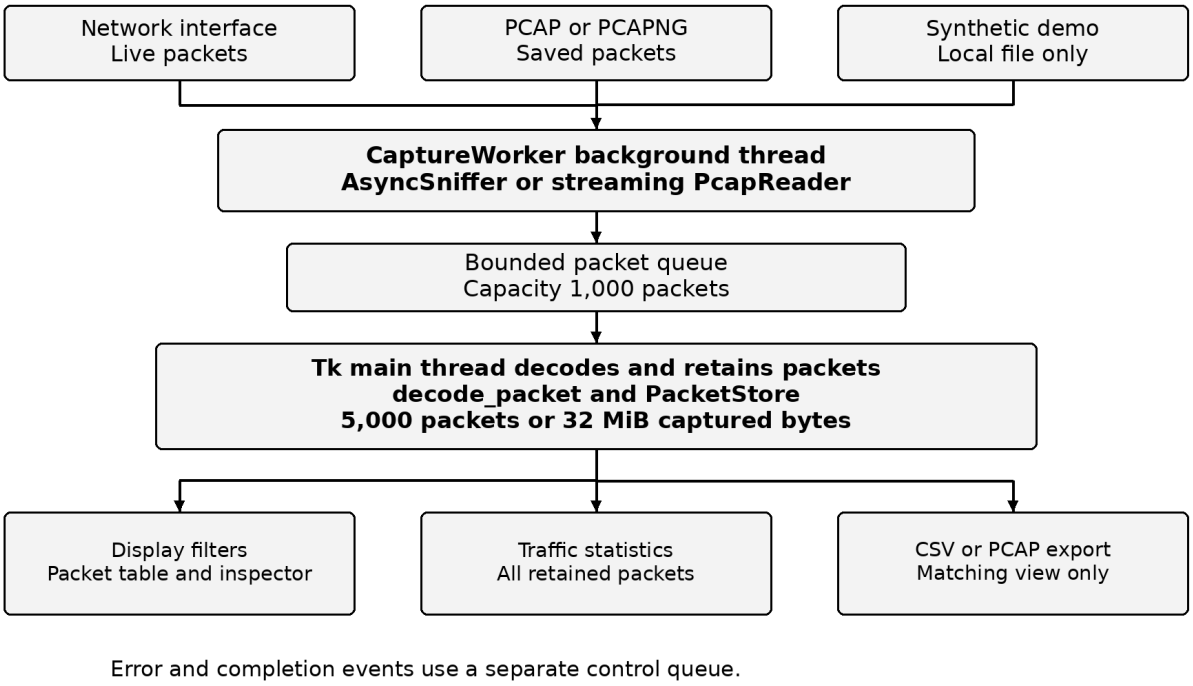


Figure 1 Implemented system architecture

The packet queue carries data. A separate event queue carries errors and completion messages. Export takes a snapshot of the current matching records; the statistics panel summarizes all retained records. The synthetic demo enters through the saved-file path and follows the same decoding process as an imported capture.

Module and Data Design

Module	Responsibility
main.py	Parse terminal arguments or launch the desktop application
analyzer/core.py	Decode packets, apply filters, hold records, calculate statistics and export files
analyzer/capture.py	Run live capture or streaming file reading in a worker thread
analyzer/gui.py	Provide controls, table, inspector, statistics and event polling
analyzer/demo.py	Construct the deterministic synthetic packet dataset

Packet record

Field	Meaning
number and timestamp	Session sequence number and capture timestamp
source and destination	IP addresses, ARP protocol addresses or link addresses
protocol and transport	Exclusive display label and separate TCP/UDP transport
source and destination port	Integer ports when a TCP/UDP layer is available
length and ip_version	Captured packet bytes and IPv4/IPv6 classification
info and packet	Short readable summary and the retained Scapy packet object

Storage and ownership

PacketStore is owned by the GUI thread and numbers retained records consecutively. The code retains a Scapy packet object so a user can inspect layers or export the captured representation later. Large decoded text dumps are generated only when a packet is selected; the application does not store one full text dump per packet.

The data is held in memory for the current session. There is no database schema or login system in this scope. A new session clears the previous retained set after confirmation. Saved captures and CSV files provide persistence when the user chooses to export them.

Chapter 6 Packet Processing Algorithm

The worker receives packets without updating widgets. The main thread periodically drains a bounded amount of queue work, decodes metadata and updates the current view. A completion event is handled after queued packets have been processed.

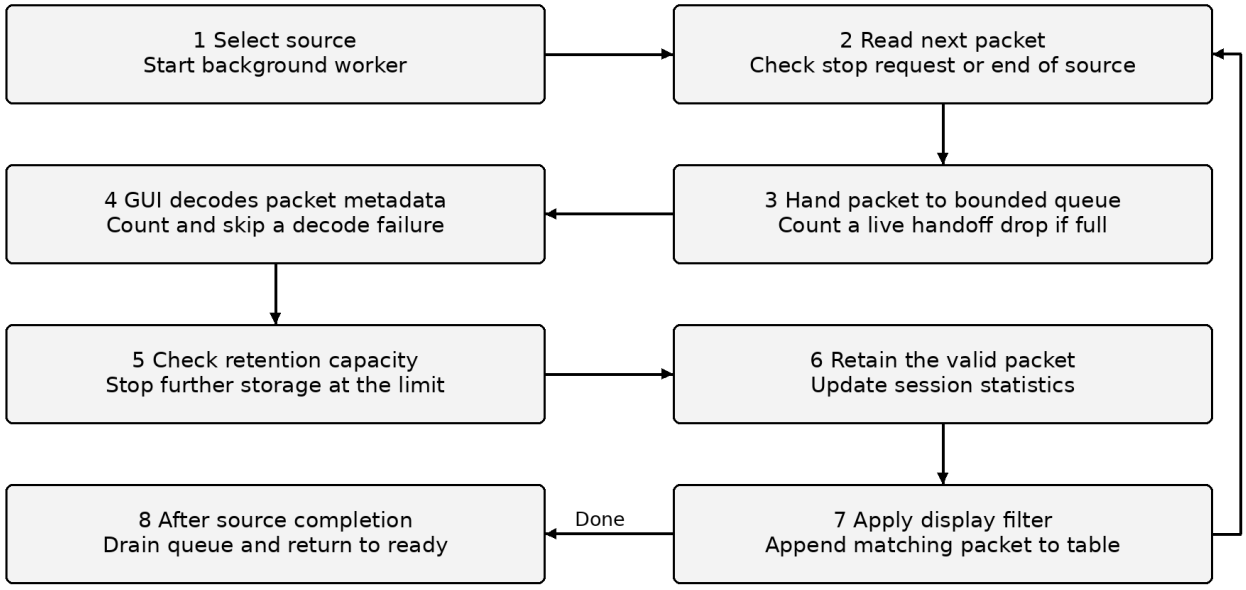


Figure 2 Packet processing and completion flow

A failed decode increments a skip counter. Once the session's packet or captured-byte limit is reached, no further packets are retained and the worker is asked to stop. File reading blocks briefly for queue space; live capture uses a nonblocking handoff and records queue drops.

Filtering and Export Algorithms

Display filtering

1. Validate the protocol choice, IP address or CIDR expression, and optional port number.
2. Check the protocol. TCP and UDP match the transport field, while DNS matches the display label.
3. If an IP condition exists, accept a packet when either endpoint belongs to the requested network.
4. If a port condition exists, require either transport endpoint to equal it.
5. If text is supplied, search the displayed metadata without case sensitivity.
6. Keep a packet only when every supplied condition succeeds.

Demo display condition	Matching packets
No filter	37
DNS	8
UDP including DNS over UDP	12
IPv6 or 2001:db8::/32	5

Demo display condition	Matching packets
DNS with port 53 and search portal.example	2

CSV and PCAP export

The export operation snapshots the matching records. CSV writes metadata columns, uses explicit UTF-8 encoding and protects packet-derived strings from being interpreted as spreadsheet formulas. PCAP writes the retained packet bytes and timestamps through Scapy. Both operations first write a temporary file in the destination directory and replace the destination after success.

Classic PCAP has one link type. The exporter checks link compatibility before writing and rejects an incompatible mixed set rather than silently combining different link encodings. PCAPNG import can carry richer source data, but classic PCAP output does not preserve every PCAPNG metadata option. CSV remains an alternative for a mixed set. [9]

Complexity

Applying a display filter examines each retained record once, so its work grows linearly with the retained packet count. Export also processes each selected record once, with additional cost proportional to packet bytes for PCAP. The 5,000-record session limit keeps the educational interface within a deliberate scope; it is not a throughput guarantee.

Chapter 7 Implementation

Acquisition

Live acquisition uses Scapy AsyncSniffer with store=False. A callback offers each packet to a bounded Python queue. If that queue is full, the callback increments the application queue-drop counter rather than waiting. The capture worker monitors the stop event and waits for the sniffer's stop callback to become available before stopping it, avoiding a start/stop initialization race. [1]

Offline input uses PcapReader to stream the file. It waits for available queue capacity and checks cancellation repeatedly. The underlying file is opened in a context manager, including during capture-reader initialization, so a malformed file does not leave the outer file handle open. Import errors are sent to the GUI through the control queue. [9]

Decoding and display

The decoder first identifies network addresses and then checks transport and control layers. A decoded DNS layer takes precedence for the display label while the transport remains TCP or UDP. The interface displays UTC timestamps, captured lengths, ports and a short summary. The selected packet can be inspected through Scapy's decoded field output and a preview of the first 256 captured bytes.

Tkinter widgets are changed only by the main thread. The polling callback limits each queue-drain batch by count and elapsed work time, then schedules the next poll. This avoids a long uninterrupted batch taking over the event loop. The session store enforces both packet-count and captured-byte limits. [10]

Statistics

Every retained packet receives one exclusive display label. DNS is therefore separate from UDP in the chart, even though a UDP display filter also includes DNS-over-UDP packets. Address rankings count observed source or destination entries. ARP rows use protocol addresses and other non-IP Ethernet rows can use MAC addresses. The average captured size divides retained bytes by retained packets.

Error handling

Invalid display-filter input leaves the current valid view in place and shows an explanation. Failed exports leave an existing destination intact. An unavailable interface, missing capture permission or invalid BPF filter is reported as a capture error, and offline analysis remains available. The terminal path returns a nonzero exit code when an operation fails.

Selected Source Code

These excerpts are taken from the supplied source. The complete application, including the interface and test modules, is included in the project package.

Bounded live packet handoff

```
    def _offer_live(self, packet):
        if self.stop_event.is_set():
            return
        try:
            self.packets.put_nowait(packet)
        except queue.Full:
            self.queue_drops += 1
```

Retention checks

```
    def add(self, packet: Packet) -> PacketRecord | None:
        if self.limit_reached:
            return None
        if len(self.records) >= self.max_packets:
            self.limit_reached = True
            return None
        try:
            record = decode_packet(packet, len(self.records) + 1)
        except Exception:
            self.decode_errors += 1
            return None
        if self.total_bytes + record.length > self.max_bytes:
            self.limit_reached = True
            return None
        self.records.append(record)
        self.total_bytes += record.length
        return record
```

The live callback does not touch GUI objects. The store accepts a decoded record only when capacity remains. The two counters describe different events: a queue drop occurs before analysis, while a decode skip occurs when metadata extraction fails.

Chapter 8 Installation and Demonstration

Windows setup

1. Extract the complete project ZIP into a writable folder.
2. Install Python 3.10 or later with Tcl/Tk support.
3. Double-click launch_windows.bat. The first setup creates a virtual environment and installs the pinned Scapy version.
4. Use Load demo to inspect the 37 synthetic packets without capture-driver privileges.
5. For live capture, install Npcap, confirm its capture permissions and select the appropriate interface. Administrator access may be required by the Npcap installation settings.

Manual commands

```
py -3 -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
.venv\Scripts\python.exe main.py --demo
```

On Linux or macOS, create the environment with python3 and use .venv/bin/python. A separate Tkinter package may be needed by the operating system. The terminal analyzer does not need a display.

Demonstration sequence

1. Show the retained-packet count and captured-byte total after loading the demo.
2. Select packet 3 and explain the DNS query for portal.example.
3. Apply DNS, show eight matching packets, then inspect the hexadecimal view.
4. Export the matching view as CSV and PCAP, and reopen the PCAP in Wireshark.
5. Open Traffic statistics and explain the exclusive protocol labels.

Capture filters and display filters

The BPF field applies to live acquisition and accepts expressions such as tcp or udp port 53. The desktop display controls operate on retained packets. Wireshark display expressions such as tcp.port == 443 use a different syntax. Confusing these two filter types is a common setup error; the report's Wireshark chapter lists comparable examples.

Chapter 9 Test Design

Verification uses a deterministic synthetic dataset so that counts and fields can be checked repeatedly. Automated cases exercise behavior that could affect the correctness of a result or the integrity of an exported file. A separate GUI script operates the real desktop widgets and can save screenshots.

Test group	Checks and expected behavior
Protocol decoding	IPv4 TCP fields, IPv6 ICMP, DNS names, ARP and unknown frames
Classification	DNS preserves its transport; port 443 alone is not labeled HTTPS
Filter validation	Exact IP, IPv6 CIDR, endpoint ports, AND conditions and invalid input
Retention and statistics	Packet/byte limits, decode skips, byte totals and out-of-order timestamps
File integrity	CSV subset, PCAP byte/timestamp round trip, PCAPNG input and corrupt files
Concurrency	Offline queue backpressure, cancellation and application queue-drop counts
Failure behavior	Live startup error reaches the control queue; incompatible export preserves destination
Terminal and GUI	CLI results and errors; desktop loading, selection, filter/reset and export actions

Execution environment

The automated run used Linux, Python 3.12.14 and Scapy 2.8.0. The GUI was exercised on a virtual graphical display. The host restricted network-interface discovery, so the optional test adapter disabled Scapy interface and route enumeration. Real Scapy parsing, file reading, export code and Tkinter widgets remained in use. No successful live capture was simulated.

Independent check

TShark 4.2.2, the Wireshark command-line analyzer, read the same PCAP independently. Its frame counts for ten expressions were compared with known expectations. Captured lengths, source/destination addresses and available TCP/UDP ports were compared for every frame. This gives evidence beyond checking the project against its own parser. [11]

Evidence files are docs/test_results.txt, docs/gui_results.txt, docs/independent_verification.json and docs/tshark_fields.csv. The final local check still needed is live capture and Windows driver/launcher behavior on the team's laptop.

Verified Results

All 29 automated cases passed, and the desktop-widget checks passed. Independent offline verification agreed on ten expected filter counts and the compared metadata for all 37 frames. These observations describe the bundled synthetic capture; they do not establish live throughput or lossless capture.

Measurement	Observed value
Retained packets	37
Captured bytes	2,384 bytes
Capture timestamp span	4.500 seconds
Average captured packet size	64.43 bytes
IPv4 packets / IPv6 packets / Non-IP frames	29 / 5 / 3

Exclusive display label	Packets	Share
TCP	14	37.8%
DNS	8	21.6%

Exclusive display label	Packets	Share
ICMP	4	10.8%
UDP excluding DNS	4	10.8%
ARP	2	5.4%
ICMPv6	2	5.4%
IPv4	1	2.7%
IPv6	1	2.7%
OTHER	1	2.7%

Interpretation

The UDP display filter returns 12 packets because it includes eight DNS-over-UDP packets and four other UDP datagrams. The chart gives the DNS packets their own label, so its UDP bar is four. Both results are correct under their stated definitions. Rounded percentage shares may not sum to exactly 100 percent.

The 4.500-second span comes from synthetic capture timestamps. It is not the program's measured processing time. Scapy's re-exported PCAP preserved the checked packet bytes and timestamps. The Wireshark GUI and live capture on Windows were not exercised in this environment.

Chapter 10 Packet Explorer Output

Figure 3 shows the real desktop application after the bundled capture has loaded. Packet 3 is selected. The source and destination fields identify the endpoints, while the protocol and port columns show DNS carried over UDP. The lower inspector displays the selected packet's decoded layers.

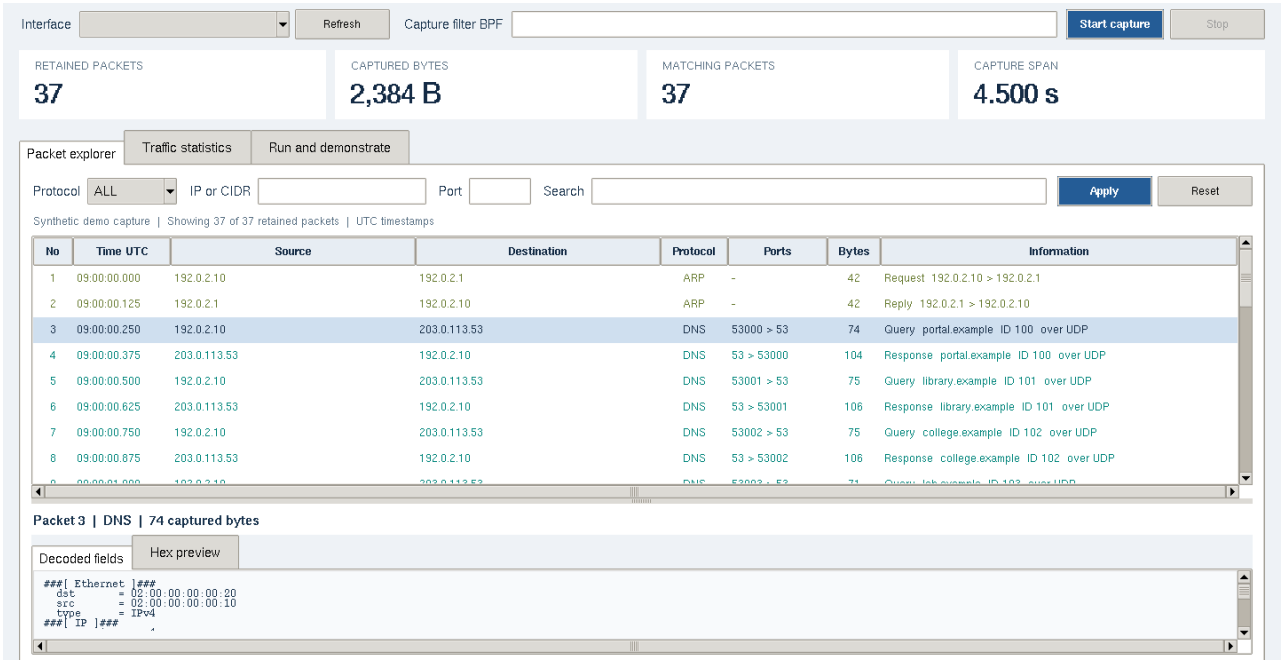


Figure 3 Actual packet explorer using the synthetic demo capture

Visible result	Meaning
37 retained and 37 matching	No display condition is active
2,384 captured bytes	Sum of captured packet lengths
Packet 3 DNS	Query from 192.0.2.10 to 203.0.113.53
53000 to 53	UDP source and destination ports

The interface selector is empty in this screenshot because interface discovery was disabled in the restricted test host. The screenshot records offline operation; it is not evidence of a live network capture. Original full-resolution screenshots are included with the project.

DNS and Hexadecimal Output

The DNS condition reduces the table to eight matching packets while the session still contains 37 retained packets. The export buttons save this matching set. The selected packet can also be examined as bytes without changing the original capture.

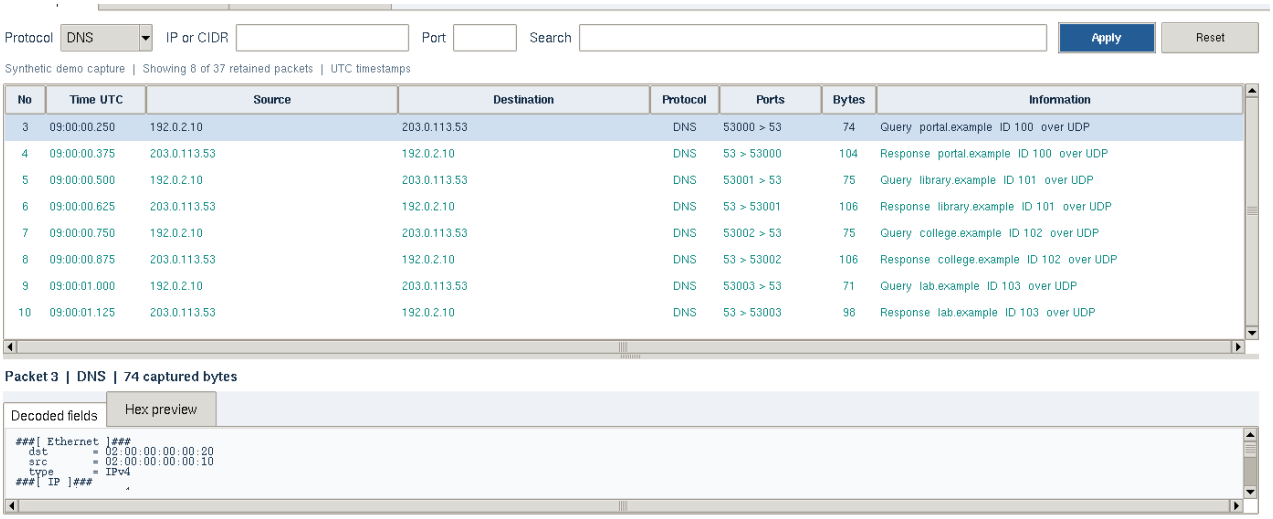


Figure 4 Actual DNS-filtered view with eight matching packets

Selected packet byte preview

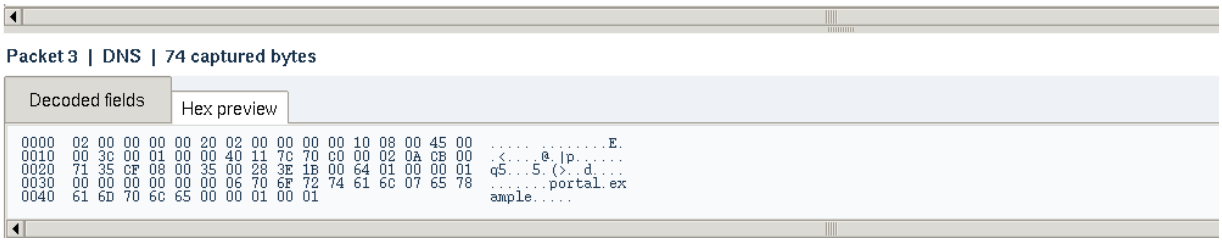


Figure 5 Enlarged crop of the actual hexadecimal inspector

Only the first 256 captured bytes are previewed for a large packet. Exported PCAP data retains the captured packet, rather than the shortened display preview.

Traffic Statistics Output

The statistics tab summarizes the complete retained session, even if a display filter is active in the explorer. Figure 6 shows exclusive protocol bars and the most frequently observed source and destination addresses from the synthetic capture.

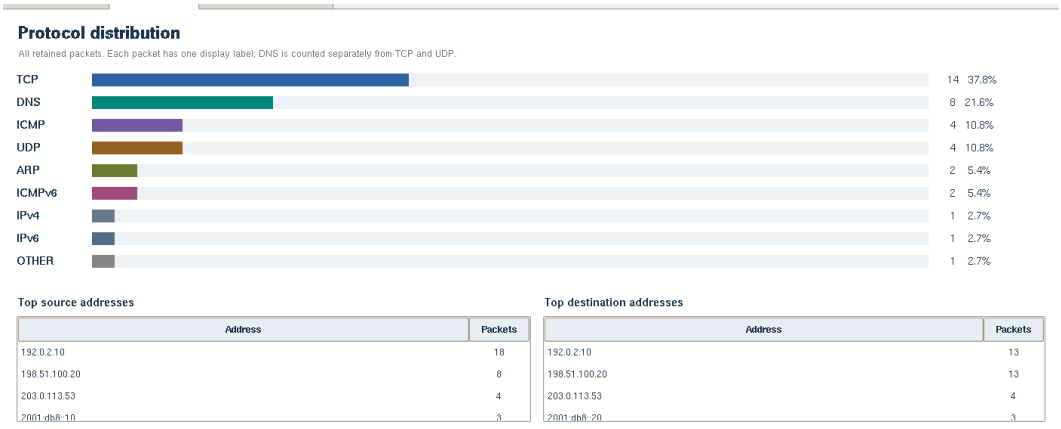


Figure 6 Actual traffic statistics for the 37-packet demo

Reading the chart

TCP is the largest exclusive label with 14 packets. DNS contributes eight packets. ICMP and UDP excluding DNS each contribute four. ARP and ICMPv6 each contribute two. The remaining IPv4, IPv6 and other Ethernet examples contribute one packet each.

The top-source list contains 192.0.2.10 with 18 observations. Address counts include the addressing convention used for each retained frame. A non-IP Ethernet frame can contribute a MAC address instead of an IP address. The figures reflect this dataset only; they are not a diagnosis of a real network.

The summary separates captured bytes, average packet size and timestamp span. These measures help explain packet composition but do not measure link capacity, network utilization or end-to-end application response time.

Chapter 11 Wireshark Verification

Completed independent verification

TShark 4.2.2 read samples/demo_capture.pcap and produced the expected results in the table below. For every frame, its captured length and decoded addresses matched the project. Available TCP/UDP source and destination ports also matched. TShark is part of the Wireshark toolset; this was an offline command-line comparison, not a Wireshark GUI or live-capture test. [11]

Wireshark display filter	Expected	Observed
(none)	37	37
dns	8	8
udp	12	12
tcp	14	14
ip	29	29
ipv6	5	5
arp	2	2
icmp	4	4
icmpv6	2	2
dns && dns.qry.name == "portal.example"	2	2

Reproduce in Wireshark on the laptop

1. Open samples/demo_capture.pcap and confirm 37 displayed frames without a filter.
2. Apply dns and inspect frame 3. Confirm portal.example, source 192.0.2.10, destination 203.0.113.53, UDP ports 53000 and 53, and 74 captured bytes.
3. Compare udp, tcp and ipv6 counts with the table above.
4. Open samples/demo_dns.pcap and confirm eight frames. This file was exported by the implemented project.

Wireshark may display more specific protocol labels than the project. Compare actual fields and selected frames rather than assuming identical protocol-column text. Capture filters use BPF syntax, while the table above uses Wireshark display syntax.

Chapter 12 Limitations and Future Work

Current limitations

The application inspects individual captured packets. It does not perform TCP stream reassembly or IP fragment reassembly. A fragmented or incomplete application message may therefore lack a decoded higher-layer label. The parser follows Scapy's available decoding and does not validate every protocol field or checksum itself.

TLS and HTTPS application contents remain encrypted. The project can present visible network and transport metadata, but a familiar port does not prove a particular application protocol. It also does not claim intrusion detection, malware classification or reconstruction of user activity.

Live visibility depends on the selected interface, driver and network. A normal switched-network capture does not automatically expose other devices' unicast traffic. The application queue-drop counter reports its own handoff overflow; it does not measure all packet loss at the NIC, kernel or capture driver.
Retention is limited to 5,000 packets or 32 MiB of captured bytes per session. Scapy objects and GUI records add memory beyond the captured-byte total. Classic PCAP export has one link type and rejects incompatible mixed sets. Export does not preserve every metadata option from PCAPNG.

Target laptop checks

- Verify Python/Tkinter installation and the Windows launcher on the actual laptop.
- Install Npcap and confirm the permitted interface and capture privileges.
- Run a short authorized live session, check Start/Stop, and reopen its PCAP in Wireshark.

Future improvements

Useful extensions include PCAPNG export with interface metadata, capture-file rotation, a virtualized table for larger sessions and configurable retention. Conversation grouping could organize related endpoints. A separate benchmark could measure packet rates and operating-system capture drops on documented hardware. TCP reassembly or encrypted-traffic handling would require additional design and tests rather than being inferred from the present implementation.

Chapter 13 Conclusion

The delivered project implements a Python desktop packet analyzer with a separate terminal path. It accepts saved captures and provides a live-capture path for supported target systems. The interface presents addresses, ports, protocol labels, timestamps and lengths. Students can inspect decoded fields, view a byte preview, apply compound filters and export the matching set.
The design makes the processing steps visible. A worker performs input operations, a bounded queue hands packets to the main thread and a session store enforces retention. Display filters leave the retained source set intact. Exports use a temporary file and replace the destination after success. The smaller module structure lets the team explain how a capture becomes a row, a chart entry and an exported record.
Verification supports the offline behavior described in this report. All 29 automated cases passed. The real desktop interface passed loading, selection, filtering and export checks. TShark independently confirmed ten filter counts and the compared metadata across all 37 synthetic frames. The demo contained 2,384 captured bytes, and PCAP round-trip checks preserved the checked bytes and timestamps.
These results support a repeatable Computer Networks demonstration. They do not establish live-capture performance. Windows launcher behavior, Npcap configuration and a short live session still need a check on the target laptop. The project package includes those setup steps, the source, sample captures, screenshots and test evidence so the team can continue from a concrete implementation.

Delivered materials

- Readable Python source for the desktop application and terminal analyzer.
- Synthetic PCAP input and exported DNS-only CSV and PCAP examples.
- Automated tests, GUI check script and independent verification evidence.
- Editable Word report and a matching PDF edition.
- Windows launcher, installation guide, team-detail fields and viva notes.

Submission preparation

Complete the register numbers, additional team members, guide details, submission date and required signatures in the Word edition. Keep the test-scope statements consistent with any additional checks completed on the team's laptop. The college's prescribed certificate can replace the unsigned template in the front matter.

References

[1] Scapy contributors. Usage and packet sniffing.
https://scapy.readthedocs.io/en/stable/usage.html

[2] Scapy contributors. Download and Installation.
https://scapy.readthedocs.io/en/stable/installation.html
[3] Wireshark Foundation. Wireshark User Guide Introduction.
https://www.wireshark.org/docs/wsug_html_chunked/ChapterIntroduction.html
[4] J Postel. Internet Protocol RFC 791 September 1981.
https://www.rfc-editor.org/rfc/rfc791.html
[5] S Deering and R Hinden. Internet Protocol Version 6 RFC 8200 July 2017.
https://www.rfc-editor.org/rfc/rfc8200.html
[6] W Eddy editor. Transmission Control Protocol RFC 9293 August 2022.
https://www.rfc-editor.org/rfc/rfc9293.html
[7] J Postel. User Datagram Protocol RFC 768 August 1980.
https://www.rfc-editor.org/rfc/rfc768.html
[8] P Mockapetris. Domain Names Implementation and Specification RFC 1035 November 1987.
https://www.rfc-editor.org/rfc/rfc1035.html
[9] Scapy contributors. Scapy utility API PcapReader and PcapWriter.
https://scapy.readthedocs.io/en/stable/api/scapy.utils.html
[10] Python Software Foundation. Tkinter Python interface to Tcl Tk.
https://docs.python.org/3/library/tkinter.html
[11] Wireshark Foundation. TShark manual page.
https://www.wireshark.org/docs/man-pages/tshark.html

Web documentation accessed on 27 October 2026. Protocol references identify the foundational specifications; later updates may apply to particular deployments. The project-specific observations are supported by the supplied source and evidence files.

Appendix Demonstration and Viva

Terminal commands

```
python main.py --analyze samples/demo_capture.pcap
python main.py --analyze samples/demo_capture.pcap --protocol DNS
python -m unittest discover -s tests -t . -v
python -m tests.gui_smoke
```

Use the virtual environment Python created during setup. The GUI check requires a graphical desktop; optional screenshot capture also requires Pillow.

Short viva answers

What is a packet A unit of network data with headers and usually a payload.
Why use Scapy It supplies capture, protocol decoding and capture-file functions to Python.
IP address versus port The IP address identifies a network endpoint; the port identifies a transport endpoint used by an application.
Why does DNS have its own label The display gives a decoded DNS layer precedence while keeping UDP or TCP as its transport.
Capture versus display filter A capture filter affects acquisition; a display filter selects among retained packets.
Why use a worker thread It keeps input waits outside Tkinter's event loop and sends packets through a bounded queue.
What was independently verified TShark confirmed ten filter counts and the compared metadata for all 37 demo frames.

Simple presentation explanation

Our project captures or opens network packets and shows their addresses, ports, protocols and sizes. Users can filter packets, inspect headers and bytes, view statistics and save matching results. The demo is repeatable, and Wireshark provides an independent way to check the saved capture.
A longer English and Tanglish explanation is supplied in docs/VIVA_GUIDE.md.